🎯 Project: Data Backup System

Internship Project Documentation

Project Objective

The objective of this project is to develop a menu-driven Java application that creates, stores, restores, imports, and manages backups of application data. The system supports binary serialization, GZIP compression, CSV and JSON formats, backup version tracking, and deletion of outdated backup files.

Technologies Used

Java, Java I/O, Object Serialization, GZIP, Collections Framework, OpenCSV, org.json, and Eclipse IDE.

Class Overview

1. BackupData.java

Represents an individual backup with a unique ID, creation time, description, and key–value data; provides methods to save the backup in binary, compressed, CSV, and JSON formats.

2. BackupManager.java

Coordinates backup creation, file restoration, CSV/JSON imports, version listing, and cleanup of old backup files.

3. Main.java

Provides the console-based menu, accepts user input, invokes BackupManager methods, and displays results and error messages.

BackupManager.java — Methods and Their Functions

Method	Purpose (one line)
BackupManager()	Initializes the backup manager and creates the required directories.
createDirectories()	Creates the backups/ and versions/ directories if they do not already exist.
createVersion(BackupData backup)	Saves a separate serialized version file identified by the backup UUID.
createBackup(String description, Map<String, Object> data)	Creates a backup, exports it in binary, GZIP, CSV, and JSON formats, saves a version, and returns its ID.
restoreFromBinary(String filePath)	Deserializes a .dat file and returns the restored BackupData object.
restoreFromCompressed(String filePath)	Decompresses and deserializes a .dat.gz file into a BackupData object.
importFromCSV(String filePath)	Reads a CSV file and returns a map of keys and typed values.

importFromJSON(String filePath)	Reads a JSON file and returns its contents as a JSONObject.
listAllVersions()	Loads saved version files and returns backups sorted by creation time, newest first.
cleanupOldBackups(int keepDays)	Deletes files in the backups/ directory older than the specified number of days.

Main.java — Menu Options

1. Create Backup 2. Restore Binary Backup 3. Restore Compressed Backup 4. Import CSV
5. Import JSON 6. List Backup Versions 7. Cleanup Old Backups 8. Exit

Code:

```
package BackupData;


import java.io.*;

import java.nio.file.*;

import java.util.*;

import java.util.zip.*;

import java.text.SimpleDateFormat;

import com.opencsv.CSVWriter;

import java.io.FileWriter;

import org.json.JSONObject;


//Backup data class to store backup attributes and functionalities

public class BackupData implements Serializable{

    private static final long serialVersionUID=1;

    //Stores backupId

    private String backupId;

    //Stores the time at which backup was created

    private Date backupTime;

    //HashMap containing data

    private Map<String,Object> data;

    //Description of backup

    private String description;
```

```
    //Parameterized constructor
    public BackupData(String description,Map<String,Object> data) {
            this.backupId=UUID.randomUUID().toString();
            backupTime=new Date();
            this.description=description;
            this.data=new HashMap<>(data);
    }

    //Saves backup as a binary file
    public void saveToBinaryFile(String filePath) throws IOException {
            try(
                    ObjectOutputStream oos=new ObjectOutputStream(new FileOutputStream(filePath))){
                    oos.writeObject(this);
                    oos.close();
            }

    }

    //Saves backup as a compressed file
    public void saveToCompressedFile(String filePath) throws IOException {
            try (
                    ObjectOutputStream oos=new ObjectOutputStream(new GZIPOutputStream( new
FileOutputStream(filePath)))){
                    oos.writeObject(this);
                    oos.close();
            }

    }

    //Saves backup as a CSV file
    public void exportToCSV(String filePath) throws IOException {
```

```
                try (CSVWriter csvWriter =
                        new CSVWriter(new FileWriter(filePath))) {

                    for (Map.Entry<String, Object> entry : data.entrySet()) {

                        String[] set = {
                            entry.getKey(),
                            entry.getValue().toString(),
                            entry.getValue().getClass().getSimpleName()
                        };

                        csvWriter.writeNext(set);
                    }
                }
            }

//Saves backup as a JSON file
public void exportToJSON(String filePath) throws IOException{

        try (FileWriter fw=new FileWriter(filePath)){
                JSONObject json = new JSONObject();

            for (Map.Entry<String, Object> entry : data.entrySet()) {

                JSONObject item = new JSONObject();

                item.put("value", entry.getValue());
                item.put("type",
                        entry.getValue().getClass().getSimpleName());
```

```
                    json.put(entry.getKey(), item);
                }


                fw.write(json.toString(4));


                System.out.println("JSON exported successfully!");
            fw.close();
             }
    }

    //Getter methods
            public String getBackupId() {
                    return backupId;
            }
            public Date getBackupTime() {
                    return backupTime;
            }
            public Map<String, Object> getData() {
                    return data;
            }
            public String getDescription() {
                    return description;
            }
            @Override
    public String toString() {
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        return String.format("Backup ID: %s\nTime: %s\nDescription: %s\nData Items: %d",
                        backupId, sdf.format(backupTime), description, data.size());
    }
}
package BackupData;
```

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.zip.*;
import java.text.SimpleDateFormat;
import org.json.JSONObject;
import com.opencsv.CSVReader;
import com.opencsv.exceptions.CsvValidationException;

//Backup Manager class containing all the utility methods
public class BackupManager {
                private static final String BACKUP_DIR = "backups/";
                private static final String VERSION_DIR = "versions/";
                public BackupManager() {
                   createDirectories();
                }

                //Creating directories where backup would be stored
                private void createDirectories() {
                   new File(BACKUP_DIR).mkdirs();
                   new File(VERSION_DIR).mkdirs();
                }

                //Creating versions of backup
                private void createVersion(BackupData backup) throws IOException {
                   String versionFile = VERSION_DIR + "version_" + backup.getBackupId() + ".dat";
                   backup.saveToBinaryFile(versionFile);
                }

                //Creating a new backup
                public String createBackup(String description, Map<String, Object> data) throws IOException {
```

```
        BackupData backup = new BackupData(description, data);

        // Save in multiple formats
        String timestamp = new SimpleDateFormat("yyyyMMdd_HHmmss").format(new Date());
        String baseName = BACKUP_DIR + "backup_" + timestamp;

        // Binary format
        backup.saveToBinaryFile(baseName + ".dat");

        // Compressed format
        backup.saveToCompressedFile(baseName + ".dat.gz");

        // CSV format
        backup.exportToCSV(baseName + ".csv");

        backup.exportToJSON(baseName + ".json");

        // Create version
        createVersion(backup);

        System.out.println("Backup created successfully!");
        System.out.println("Files created:");
        System.out.println("  • " + baseName + ".dat");
        System.out.println("  • " + baseName + ".dat.gz");
        System.out.println("  • " + baseName + ".csv");
        System.out.println("  • " + baseName + ".json");

        return backup.getBackupId();
    }

    //Restore data from a saved binary file
```

```
    public BackupData restoreFromBinary(String filePath) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream(filePath))) {
            return (BackupData) ois.readObject();
        }
    }

    //Restore data from a saved compressed file
    public BackupData restoreFromCompressed(String filePath) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(
            new GZIPInputStream(new FileInputStream(filePath)))) {
            return (BackupData) ois.readObject();
        }
    }

    //Restore data from a saved CSV file
    public Map<String, Object> importFromCSV(String filePath)
            throws IOException, CsvValidationException {

        Map<String, Object> result = new HashMap<>();

        try (CSVReader reader =
            new CSVReader(new FileReader(filePath))) {
    String[] set;

            while ((set = reader.readNext()) != null) {

                String key = set[0];
                String value = set[1];
                String type = set[2];
```

```
    Object obj;

            switch (type) {
                case "Integer":
                    obj = Integer.parseInt(value);
                    break;
                case "Double":
                    obj = Double.parseDouble(value);
                    break;
                case "Boolean":
                    obj = Boolean.parseBoolean(value);
                    break;
                case "String":
                    obj = value;
                    break;
                default:
                    throw new IllegalArgumentException(
                        "Unsupported type: " + type);
            }

            result.put(key, obj);
        }
    }

    return result;
  }

//Restore data from a saved JSON file
  public JSONObject importFromJSON(String filePath)
        throws IOException {
```

```
        String content = Files.readString(Path.of(filePath));

        return new JSONObject(content);
    }

    //Lists all versions of the backup created
    public List<BackupData> listAllVersions() throws IOException, ClassNotFoundException {
        List<BackupData> versions = new ArrayList<>();
        File versionDir = new File(VERSION_DIR);

        if(versionDir.exists()) {
            File[] files = versionDir.listFiles((dir, name) -> name.endsWith(".dat"));
            if(files != null) {
                for(File file : files) {
                    versions.add(restoreFromBinary(file.getPath()));
                }
            }
        }

        // Sort by backup time (newest first)
        versions.sort((b1, b2) -> b2.getBackupTime().compareTo(b1.getBackupTime()));
        return versions;
    }

    //Removes all existing backups
    public void cleanupOldBackups(int keepDays) {
        File backupDir = new File(BACKUP_DIR);
        if(!backupDir.exists()) return;

        File[] files = backupDir.listFiles();
        if(files == null) return;
```

```
                long cutoffTime = System.currentTimeMillis() - (keepDays * 24L * 60 * 60 * 1000);

                for(File file : files) {
                    if(file.lastModified() < cutoffTime) {
                        if(file.delete()) {
                            System.out.println("Deleted old backup: " + file.getName());
                        }
                    }
                }
            }
}
package BackupData;

import java.util.*;
import java.io.*;
import org.json.JSONObject;
import com.opencsv.exceptions.CsvValidationException;



//Contains the main function to implement all the functionalities of BackupManager class
public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        BackupManager manager = new BackupManager();

        int choice = 0;

        while (choice != 8) {
```

```
System.out.println("\n========== DATA BACKUP SYSTEM ==========");
System.out.println("1. Create Backup");
System.out.println("2. Restore Backup from Binary File");
System.out.println("3. Restore Backup from Compressed File");
System.out.println("4. Import Data from CSV");
System.out.println("5. Import Data from JSON");
System.out.println("6. List All Backup Versions");
System.out.println("7. Cleanup Old Backups");
System.out.println("8. Exit");
System.out.print("Enter your choice: ");

try {
    choice = Integer.parseInt(sc.nextLine());

    switch (choice) {

        case 1:
            //Implement backup data creation
            System.out.print("Enter backup description: ");
            String description = sc.nextLine();

            //To store data of backups
            Map<String, Object> data = new HashMap<>();

            System.out.print("Enter number of data items: ");
            int n = Integer.parseInt(sc.nextLine());

            for (int i = 0; i < n; i++) {

                System.out.print("Enter key: ");
```

```
                String key = sc.nextLine();

                System.out.print(
                    "Enter data type (String/Integer/Double/Boolean): ");
                String type = sc.nextLine();

                System.out.print("Enter value: ");
                String value = sc.nextLine();

                Object obj;

                switch (type.toLowerCase()) {

                    case "integer":
                        obj = Integer.parseInt(value);
                        break;

                    case "double":
                        obj = Double.parseDouble(value);
                        break;

                    case "boolean":
                        if (!value.equalsIgnoreCase("true") &&
                            !value.equalsIgnoreCase("false")) {
                            throw new IllegalArgumentException(
                                "Boolean must be true or false");
                        }
                        obj = Boolean.parseBoolean(value);
                        break;

                    case "string":
```

```
                    obj = value;
                    break;

                default:
                    throw new IllegalArgumentException(
                        "Unsupported data type: " + type);
                }

                data.put(key, obj);
            }

            String backupId = manager.createBackup(description, data);
            System.out.println("Backup ID: " + backupId);
            break;

        case 2:
            //Restores data from binary file
            System.out.print("Enter binary file path: ");
            String binaryPath = sc.nextLine();

            BackupData binaryBackup =
                manager.restoreFromBinary(binaryPath);

            System.out.println("\nBackup restored successfully!");
            System.out.println(binaryBackup);
            System.out.println("Data: " + binaryBackup.getData());
            break;

        case 3:
            //Restore data from compressed file
            System.out.print("Enter compressed file path: ");
```

```
            String compressedPath = sc.nextLine();

            BackupData compressedBackup =
                manager.restoreFromCompressed(compressedPath);

            System.out.println("\nBackup restored successfully!");
            System.out.println(compressedBackup);
            System.out.println("Data: " + compressedBackup.getData());
            break;

        case 4:
                //Restore data from CSV file
            System.out.print("Enter CSV file path: ");
            String csvPath = sc.nextLine();

            Map<String, Object> csvData =
                manager.importFromCSV(csvPath);

            System.out.println("\nCSV imported successfully!");
            System.out.println("Imported Data: " + csvData);
            break;

        case 5:
                //Restore data from JSON file
            System.out.print("Enter JSON file path: ");
            String jsonPath = sc.nextLine();

            JSONObject jsonData =
                manager.importFromJSON(jsonPath);

            System.out.println("\nJSON imported successfully!");
```

```
                System.out.println(jsonData.toString(4));
                break;

            case 6:
                List<BackupData> versions =
                    manager.listAllVersions();

                if (versions.isEmpty()) {
                    System.out.println("No backup versions found.");
                } else {
                    System.out.println("\n========== BACKUP VERSIONS ==========");

                    for (BackupData backup : versions) {
                        System.out.println(backup);
                        System.out.println("Data: " + backup.getData());
                        System.out.println("----------------------------");
                    }
                }
                break;

            case 7:
                    //Removes all old backups
                System.out.print(
                    "Enter number of days to retain backups: ");

                int days = Integer.parseInt(sc.nextLine());

                if (days < 0) {
                    System.out.println("Days cannot be negative.");
                    break;
                }
```

```
                    manager.cleanupOldBackups(days);
                    System.out.println("Backup cleanup completed.");
                    break;

                case 8:
                    System.out.println("Exiting Data Backup System...");
                    break;

                default:
                    System.out.println("Invalid choice! Try again.");
            }

        } catch (NumberFormatException e) {
            System.out.println("Invalid number entered.");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println("Error restoring backup: " + e.getMessage());
        } catch (CsvValidationException e) {
            System.out.println("CSV error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid input: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }

    sc.close();
  }
}
```

Output:

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 1

Enter backup description: User Database Backup

Enter number of data items: 3

Enter key: users_count

Enter data type (String/Integer/Double/Boolean): String

Enter value: 1500

Enter key: last_backup

Enter data type (String/Integer/Double/Boolean): String

Enter value: 2024-10-12

Enter key: backup_size_mb

Enter data type (String/Integer/Double/Boolean): Double

Enter value: 2450.75

JSON exported successfully!

Backup created successfully!

Files created:

- backups/backup_20261104_220033.dat
- backups/backup_20261104_220033.dat.gz
- backups/backup_20261104_220033.csv
- backups/backup_20261104_220033.json

Backup ID: 00768169-b13c-4dd8-bea2-aecacb9d8a01

========== DATA BACKUP SYSTEM ==========

1. Create Backup
2. Restore Backup from Binary File
3. Restore Backup from Compressed File
4. Import Data from CSV
5. Import Data from JSON
6. List All Backup Versions
7. Cleanup Old Backups
8. Exit

Enter your choice: 4

Enter CSV file path: backups/backup_20261104_220033.csv

CSV imported successfully!

Imported Data: {users_count=1500, backup_size_mb=2450.75, last_backup=2024-10-12}

========== DATA BACKUP SYSTEM ==========

1. Create Backup
2. Restore Backup from Binary File
3. Restore Backup from Compressed File
4. Import Data from CSV
5. Import Data from JSON
6. List All Backup Versions
7. Cleanup Old Backups

8. Exit

Enter your choice: 1

Enter backup description: Application Settings Backup

Enter number of data items: 5

Enter key: app_name

Enter data type (String/Integer/Double/Boolean): String

Enter value: Employee Management System

Enter key: app_version

Enter data type (String/Integer/Double/Boolean): String

Enter value: 3.1.12

Enter key: dark_mode

Enter data type (String/Integer/Double/Boolean): Boolean

Enter value: true

Enter key: storage_limit_gb

Enter data type (String/Integer/Double/Boolean): Double

Enter value: 3.2

Enter key: default_language

Enter data type (String/Integer/Double/Boolean): String

Enter value: English

JSON exported successfully!

Backup created successfully!

Files created:

- backups/backup_20261104_220303.dat
- backups/backup_20261104_220303.dat.gz
- backups/backup_20261104_220303.csv
- backups/backup_20261104_220303.json

Backup ID: 7acdaa09-a2bb-495f-8d4e-c035896eb2c0

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 1

Enter backup description: System Configuration

Enter number of data items: 2

Enter key: backup_date

Enter data type (String/Integer/Double/Boolean): String

Enter value: 2026-02-27

Enter key: backup_size_mb

Enter data type (String/Integer/Double/Boolean): Double

Enter value: 320.5

JSON exported successfully!

Backup created successfully!

Files created:

- backups/backup_20261104_220411.dat
- backups/backup_20261104_220411.dat.gz
- backups/backup_20261104_220411.csv
- backups/backup_20261104_220411.json

Backup ID: 4b2f12b0-64a6-44d8-aa34-b0745af2de58

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 2

Enter binary file path: backups/backup_20261104_220303.dat

Backup restored successfully!

Backup ID: 7acdaa09-a2bb-495f-8d4e-c035896eb2c0

Time: 2026-11-04 22:03:03

Description: Application Settings Backup

Data Items: 5

Data: {storage_limit_gb=3.2, app_name=Employee Management System, app_version=3.1.12, dark_mode=true, default_language=English}

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 3

Enter compressed file path: backups/backup_20261104_220411.dat.gz

Backup restored successfully!

Backup ID: 4b2f12b0-64a6-44d8-aa34-b0745af2de58

Time: 2026-11-04 22:04:11

Description: System Configuration

Data Items: 2

Data: {backup_size_mb=320.5, backup_date=2026-02-27}

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 5

Enter JSON file path: backups/backup_20261104_220303.json

JSON imported successfully!


```
{
    "storage_limit_gb": {
        "type": "Double",
        "value": 3.2
    },
    "app_name": {
        "type": "String",
        "value": "Employee Management System"
    },
```

```
    "app_version": {
        "type": "String",
        "value": "3.1.12"
    },
    "dark_mode": {
        "type": "Boolean",
        "value": true
    },
    "default_language": {
        "type": "String",
        "value": "English"
    }
}
```


========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 6

========== BACKUP VERSIONS ==========

Backup ID: 4b2f12b0-64a6-44d8-aa34-b0745af2de58

Time: 2026-11-04 22:04:11

Description: System Configuration

Data Items: 2

Data: {backup_size_mb=320.5, backup_date=2026-02-27}

Backup ID: 7acdaa09-a2bb-495f-8d4e-c035896eb2c0

Time: 2026-11-04 22:03:03

Description: Application Settings Backup

Data Items: 5

Data: {storage_limit_gb=3.2, app_name=Employee Management System, app_version=3.1.12, dark_mode=true, default_language=English}

Backup ID: 00768169-b13c-4dd8-bea2-aecacb9d8a01

Time: 2026-11-04 22:00:33

Description: User Database Backup

Data Items: 3

Data: {users_count=1500, backup_size_mb=2450.75, last_backup=2024-10-12}

Backup ID: 9422a1b4-9e61-455e-966a-33dd628ba81e

Time: 2026-11-04 21:29:45

Description: Applications Settings Backup

Data Items: 4

Data: {app_name=Employee Management System, storage occupied=3.2, app_version=3.2.11, last_backup=2025-09-15}

Backup ID: e2ed7b38-ae69-4a83-a59f-bce824c5eb73

Time: 2026-11-04 21:27:02

Description: System Configuation Backup

Data Items: 2

Data: {data_size(in mb)=450.0, last_backup=2026-10-01}

Backup ID: f05dc14e-44f1-408d-bb1a-bf5508dd54fa

Time: 2026-11-04 21:23:29

Description: User Database Backup

Data Items: 3

Data: {users_count=1500, database_size_mb=2450.75, last_backup=2024-01-15}

Backup ID: 873d2432-fb6c-4617-adc5-07c7cf45bc42

Time: 2026-11-04 21:17:35

Description: user data backup

Data Items: 3

Data: {user_count=1500, database_size_mb=2450.75, last_backup=2024-01-15}

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 7

Enter number of days to retain backups: 2

Backup cleanup completed.

========== DATA BACKUP SYSTEM ==========

1. Create Backup

2. Restore Backup from Binary File

3. Restore Backup from Compressed File

4. Import Data from CSV

5. Import Data from JSON

6. List All Backup Versions

7. Cleanup Old Backups

8. Exit

Enter your choice: 8

Exiting Data Backup System...

Conclusion

This project demonstrates Java file handling, serialization, compression, external library integration, and object-oriented design through a practical backup management application.

Scope and Limitations

CSV import/export currently supports String, Integer, Double, and Boolean values. JSON import returns a JSONObject rather than a BackupData object. Cleanup affects backups/ but not versions/. The console menu does not yet include scheduling, comparison, or merging.